

1. Asymptotic Analysis

- **Asymptotic upper bound:** $f(n) = O(g(n))$ if there exist $c, n_0 > 1$ such that $f(n) \leq cg(n)$ for all $n \geq n_0$.

- **Function hierarchy:** for any constants a, b, c, d ,

$$\log^a n < n^b < 2^{cn} < n! < n^{dn}$$

- **Master theorem** - Given $T(n) = aT(n/b) + cn^k$ and $T(1) = c$,

- $T(n) = \Theta(n^k)$ if $a < b^k$.
- $T(n) = \Theta(n^k \log n)$ if $a = b^k$.
- $T(n) = \Theta(n^{\log_b a})$ if $a > b^k$.

- String concatenation is $O(n)$, where n is length of string

- For recursive algorithms, use **recurrence relations**

1. Come up with a recurrence relation for runtime $T(n)$.
2. Expand recurrence relation a few times, spot the pattern.
3. Figure out when it stops (base case).
4. Analyse the resulting sum.

- Common recurrence relations:

- $T(n) = T(n/2) + O(1) = O(\log n)$ (binary search)
- $T(n) = 2T(n/2) + O(n) = O(n \log n)$ (merge sort)
- $T(n) = T(n-1) + O(1) = O(n)$
- $T(n) = T(n-1) + O(n) = O(n^2)$
- $T(n) = 2T(n/4) + O(1) = O(\sqrt{n})$

3. Searching

- **Invariant:** show initialisation, maintenance, termination

- **Binary search:** $O(\log n)$

1. Set indices $left = 1$ and $right = n$. Maintain search interval $[left, right]$.
2. While $left \leq right$, find middle index of sub-array.
3. Compare middle element $A[mid]$ with k .
 - If $k = A[mid]$, found k . Return mid.
 - If $k > A[mid]$, then k lies in right half. Update $left = mid + 1$.
 - If $k < A[mid]$, then k lies in left half. Update $right = mid - 1$.

- **Partitioning:** Given array $A[1..n]$ and pivot index p , let $pivot$ be the original value of $A[p]$; permute A such that there exists index i such that $A[1..i-1] \leq pivot$, $pivot \leq A[i+1..n]$. $O(n)$

1. Initially set indices $j = 1$ and $i = 1$.
They mark boundaries of sub-arrays $A[2..j]$ and $A[j+1..i]$.
2. Decide whether $A[i+1]$ belongs in the left or right sub-array.
 - If $A[i+1] < A[1]$, then $A[i+1]$ belongs in left sub-array. Swap $A[i+1]$ with $A[j+1]$. Increment i, j .
 - If $A[i+1] \geq A[1]$, then $A[i+1]$ belongs in right sub-array. Increment i .

- **Quick select:** Given array $A[1..n]$, output r -th smallest element. $O(n^2)$

1. Set indices $left = 1, right = n$.
2. Run partitioning on sub-array $A[left, right]$. Let i be the resulting index of the pivot.
3. If $i = r$, then found r -th smallest integer. Return $A[i]$.
If $r < i$, then r -th smallest integer lies in left half. Update $right = i - 1$, recurse.
If $r > i$, then r -th smallest integer lies in the right half. Update $left = i + 1$, recurse.

- **Randomised quick select:** $O(n)$

1. Set indices $left = 0, right = n$.
2. Keep randomly selecting an element in $A[left, right]$ to use as a pivot, run partition on $A[left, right]$ until pivot lies in middle third.
3. If $i = k$, return $A[i]$.
If $k < i$, update $right = i$. Repeat step 2.
If $k > i$, update $left = i$. Repeat step 2.

- **Peak finding:**

1. Maintain a search interval $A[left, right]$.
2. Probe the middle. If $A[mid]$ is a peak, return mid . Else recurse on left/right sub-array.

4. Sorting

Algo	Worst case	Expected	Space	Stability
Insertion	$O(n^2)$	$O(n^2)$	$O(1)$	stable
Selection	$O(n^2)$	$O(n^2)$	$O(1)$	unstable
Merge	$O(n \log n)$	$O(n \log n)$	$O(n)$	stable
Quick	$O(n^2)$	$O(n^2)$	$O(n)$	unstable
Random quick	∞	$O(n \log n)$	$O(\log n)$	unstable

- **Insertion sort:** *shifting items until their rightful place* $O(n^2)$

1. For $i = 2, \dots, n$, exchange $A[i]$ with the entries in $A[1..i-1]$ until it is in place.

Invariant: At the start of i -th iteration, $A[1..i-1]$ is in sorted order.

- **Selection sort:** *getting it right the first time* $O(n^2)$

1. For each $i = 1, 2, \dots, n$, find minimum element of $A[i..n]$.
2. Swap it with $A[i]$.

Invariant: At the start of i -th iteration, $A[1..i-1]$ is the same as *sorted* $[1..i-1]$.

- **Merge sort:** *divide-and-conquer* $O(n \log n)$

1. Divide array into two halves.
2. Sort two sub-arrays recursively using Merge Sort.
3. Merge the two sorted sub-arrays. $O(n)$

- **Quick sort:** $O(n^2)$

1. Partition the array
2. Sort two sub-arrays recursively using Quick Sort.
3. Recombine the arrays.

- **Random partitioning:** keep randomly sample pivot until its final position lies in middle third of array. expected $O(n)$
In the worst case, split array into 2/3 and 1/3.

- **Randomised quick sort:** expected $O(n \log n)$

1. Random partition the array
2. Sort two sub-arrays recursively.
3. Recombine the arrays.

- **Bubble sort:** *compare and swap* $O(n^2)$

Invariant: At the end of j -th iteration, the biggest j items are correctly sorted in the final j position of the array

- Sort (a, b) by a then b :

1. Use any sorting algo to sort by b .
2. Use stable sorting to sort by a (preserves ordering of b).

5. Elementary Data Structures

- **Array:** read $O(1)$, insert $O(n)$, size $O(1)$
- **Linked list:** set $O(n)$, delete $O(n)$, lookup $O(n)$, size $O(1)$, append $O(1)$
- **LIFO stack:** push $O(1)$, pop $O(1)$, peek $O(1)$
- **FIFO queue:** enqueue $O(1)$, dequeue $O(1)$, peek $O(1)$
- Augmenting data structures

1. Choose an underlying data structure (array, linked list, tree)
2. Determine additional info needed (weight, rank, height)
3. Modify data structure to maintain additional info when the structure changes
4. Develop new operations

6. Binary Search Tree

- **Binary search tree** (BST) is a binary tree. Each node stores key-value pair, pointer to left & right child.
- **BST invariant:** all in left sub-tree $< key <$ all in right sub-tree
- **Minimum key:** traverse to left most node on the left subtree (from root)
- **Maximum key:** traverse to right most node on the right subtree (from root)
- **Search:** given a target, check current node's key against target. If target $\leq$ key, traverse left and repeat search. Else, traverse right and repeat search. $O(h)$
- **Insertion:** given a key k , check current node's key against k . If $k \leq$ key, insert into left sub-tree; else right sub-tree. If sub-tree is null, insert as a leaf. $O(h)$
- **Deletion:** $O(h)$

1. Leaf: delete
2. 1 child: delete, attach child to deleted node's parent.
3. 2 childs: find successor of node to delete, swap the successor with the node to delete, delete the node (now leaf)

- **Tree traversals:** $O(n)$

- Pre-order: self, left, right
- In-order: left, self, right
- Post-order: left, right, self
- Level-order: level by level

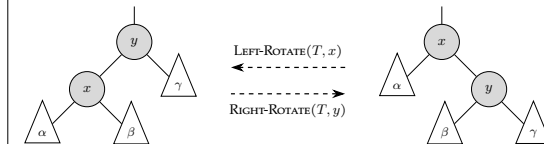
- **Size augmentation:** *size* is the number of nodes in the sub-tree rooted at that node
- **Rank:** rank of k equals number of elements smaller than k
At current node,
 - If key $= k$, then we have found k . Return size of left sub-tree.
 - If key $> k$, recurse on left sub-tree.
 - If key $< k$, recurse on right sub-tree. Whatever answer we obtain, add size of left sub-tree $+ 1$.

- **Select:** check current node's left subtree size against *rank*

- If $left.size = rank$, found the key. Return it.
- If $left.size > rank$, recurse on left sub-tree.
- If $left.size < rank$, recurse on right sub-tree.

7. AVL Tree

- **AVL invariant:** for each node, heights of its left and right sub-trees differ by at most 1.
- **Height augmentation:** *height* is number of edges on the longest path from root to leaf
 - If node is leaf, height = 0
 - If node is null, height = -1
 - Else, height = $\max(\text{left.height}, \text{right.height}) + 1$
- AVL tree with n nodes has height $O(\log n)$.
- **Left rotation, right rotation**



- **Heaviness:** **left/right heavy** if left/right subtree has larger height than right/left subtree
If one side is too tall, rotate it so that taller sub-tree moves up: left heavy $\Rightarrow$ right rotation; right heavy $\Rightarrow$ left rotation
- **Rotation order:**
 - v is left-heavy
 - * v.left is balanced: right rotation on v
 - * v.left is left-heavy: right rotation on v
 - * v.left is right-heavy: left rotation on v.left, right rotation on v
 - v is right-heavy
 - * v.right is balanced: left rotation on v
 - * v.right is right-heavy: left rotation on v
 - * v.right is left-heavy: right rotation on v.right, left rotate on v

8. Hash Table

- n : **number of keys** currently inserted into the table
- m : **table size** (i.e., size of the array)
- **Hash function** $h: U \rightarrow \{0, 1, \dots, m-1\}$ maps key k to an integer.
Element with key k is stored in slot $h(k)$.
- General flow of inserting key k into hash table of size m :
 1. Compute the hash $h(k)$ (usually involves $\text{mod } m$).
 2. If the slot at that index is empty, insert x there.
 3. Otherwise, handle collisions.
- **SUHA:** every key is equally likely to map to every slot, and keys are mapped independently

Separate Chaining

- Each slot contains a **linked list** of items, where all items have the same hash.
- Expected: lookup $O(n/m)$, insert $O(n/m)$, delete $O(n/m)$, size $O(1)$, list pairs $O(1)$
- Lookup(k): expected $O(n/m)$, worst $O(n)$
Evaluate the hash $h(k)$, look up the list at index $h(k)$.
If k exists in the list, return the value; otherwise, return null.
- Insert(k, v): expected $O(n/m)$, worst $O(n)$
Evaluate the hash $h(k)$, look up the list at index $h(k)$.
If k exists in the list, overwrite the existing value; otherwise, append (k, v) to the list.
- Delete(k): expected $O(n/m)$, worst $O(n)$
Evaluate the hash $h(k)$, look up the list at index $h(k)$.
If k exists in the list, delete it; otherwise, return null.

Open Addressing - Linear Probing

- Each slot contains only one element; any collisions will **probe a sequence of slots** until an empty one is found
- Expected: lookup $O(1)$, insert $O(1)$, delete $O(1)$, size $O(1)$, list pairs $O(1)$
- Lookup(k): expected $O(1)$, worst $O(n)$
Start at index $h(k)$, iterate until either find an empty table slot, or tried all m slots.
If found the slot where k is stored, return the value; otherwise, return null.
- Insert(k, v): expected $O(1)$, worst $O(n)$
If size = m , notify the user of an error.
Start at index $h(k)$, iterate until either find an empty table slot, or the slot that holds k .
Store (k, v) at the slot we found.
- Delete(k): When we remove a key from the table, we run the risk of breaking another “run”. Thus, when we delete a key, we remember its current index idx . Then we keep going and see if any items have hash value $\leq idx$. If it does, put that item into index idx . Repeat.

expected $O(1)$, worst $O(n)$

 - Start at index $h(k)$, iterate for at most m iterations until either find an empty table slot, or the slot that holds k .
 - If it is empty, then k does not exist in the table. Thus, we just return.
If we found the slot that holds k , delete it and set idx to remember our current index. Decrement $size$.
 - Iterate forwards until we find an element with key k' such that $h(k') \leq idx$.
Move it into slot idx . Update idx to the current index. Continue iterating.

9. Hashing Problems

- Find Most Frequently Occuring
 - Traverse the array once, maintain a hash table that records the frequency of each element encountered.
Key is the value of the element, value is the **frequency** of the element.
 - Retrieve all key-value pairs, find maximum frequency, then output all keys with that frequency.

Time complexity:

 - Expected $O(n)$, because each insert and lookup in the hash table takes expected time $O(1)$.
 - Worst-case $O(n^2)$, if all keys collide, since then each insertion or lookup takes worst-case time $O(n)$.
 - Extracting the maximum frequency takes $O(n)$ time, since we loop through all key-value pairs.
- Find Uniques:
 - Traverse the array once, while maintaining a hash table *counts* that records the frequency of each element encountered. (Similar to first half of Problem 1.)
 - Retrieve all key-value pairs, find keys with value = 1.
- Longest Sub-array with Unique Elements
 - Maintain two pointers *left* and *right*, and a hash table tracking elements in the current window.
 - Advance *right* by one. If $A[right]$ is already in the window, advance *left* until all elements in $A[left, right)$ are unique again.
 - Update the hash table to reflect the current window.

Since we advance left at most n times, and advance right at most n times. We are making at most n lookup, insert and remove operations. Hence, the expected time is $n \times O(1) = O(n)$.

10. Binary Heap

- A **binary heap** organises its nodes in the form of complete binary tree. A **complete binary tree** is a tree where
 - Every node has either 0, 1, or 2 children.
 - Every level of the tree is **full** except the last level (which does not need to be).
 - The last level is filled from left to right.
- Min heap**: The priorities of parent $\leq$ children for all subtrees. (**Binary heap invariant**)
Max heap: The priorities of parent $\geq$ children for all subtrees.
- Priority-value pairs of the heap are stored in an array. Nodes are labelled from left to right starting at index 1.
- Height of binary heap is $O(\log n)$.
- Insertion:

$O(\log n)$
 - Insert the element at index $size + 1$.
 - Bubble up**: Restore heap order by keep swapping with its parent until either it is $\geq$ than its parent node, or it is the root node.
- Peek min: Smallest element is at index 1, so return it

$O(1)$
- Decrease key: Locate node, update key, then bubble up until heap order is restored

$O(\log n)$
- Extract-Min:

$O(\log n)$
 - Swap root node with last node
 - Remove the last node
 - Bubble down**: Restore heap order by swapping with the *smaller* of its two children, until it is $\geq$ both its children, or it is leaf node.
- Heapify: construct a heap from an existing array

$O(n)$
 - Shift elements down by 1, so that array indexing starts at 1.
 - The right-most internal node has index $size/2$. Apply bubble-down at indices $size/2, \dots, 1$.